

Object Oriented Programming

Object-Oriented Programming (OOP) is a foundational paradigm in Software Engineering that structures code around **object** and **classes** rather than relying solely on functions and procedures. By modeling software as a collection of interacting entities with their own attributes and behaviors, OOP effectively mirrors real-world systems, leading to more intuitive and organized code.

Core Concepts of OOP

1. Classes and Objects
 - A class is a blueprint that define attributes (data) and methods (behavior).
 - An object is a specific instance created from a class definition, possessing its own unique state while sharing structure defined bu its class.
 - This pairing enables modular and scalable design by encapsulating data and behavior together.
2. Encapsulation
 - The bundling of data and the methods that operate on that data into a single unit (a class).
 - It protects the internal state of an object by restricting direct access, exposing only controlled interfaces (public methods).
 - In Python, this is primarily a convention using underscores: a single underscore `_` for "protected" and a double underscore `__` for "private" members.
3. Inheritance
 - Enables child classes to inherit attributes and methods from parent classes.
 - Promotes code reuse and hierarchical organization.
 - Child classes can override or extend (add new) behavior.
4. Polymorphism
 - Different classes can implement the same method in their own way.
 - Enables writing code that works across different types uniformly via a common interface.
5. Abstraction
 - Hiding implementation details, exposing only essential features.
 - Hiding internal details and exposing only what is necessary.
 - This hides implementation details and ensures consistent interfaces across related classes.
6. Composition
 - Represents has-a relationships (objects containing other objects).
 - Offers an alternative to inheritance (is-a) for building modular systems.
7. Method Overloading
 - Python does not support traditional method overloading.
 - Instead, it uses default arguments or `*args`, `**kwargs` to achieve similar behavior.
8. Class and Static Methods

Lecture 10

- Class methods operate on the class itself and not on individual instances.
- Static methods are utility functions grouped logically with the class but don't need class or instance data.

Together, these principles foster maintainable, reusable, and intuitive code that reflects real-world relationships and behaviors.

Other Programming Paradigms

As you may have gleaned from the above, a paradigm is a style that code can be organised. It consists of certain structures, features, and opinions about how common programming problems should be tackled. It is an evolving process that is happening even now!

A short list of the most popular programming styles

1. Imperative Programming

- Focuses on **how** to achieve a result through explicit step-by-step instructions.
- Involves changing program state via assignments, loops, and conditionals.
- **Procedural programming** is a subset of imperative programming that organizes code into reusable procedures/functions.
- *Examples*: C and Pascal. Python supports imperative constructs alongside OOP.
- *Pros*: Offers granular control
- *Cons*: Can become complex and unwieldy in large system.

2. Declarative Programming

- Focuses on what the program should accomplish, not how to accomplish it.
- The programmer specifies the desired result, and the system determines the execution steps.
- *Examples*: include SQL (database queries), HTML/CSS (web structure and styling), and configuration files.
- Benefits include improved readability and reduced complexity by abstracting control flow.

3. Functional Programming

- Treats computation as the evaluation of mathematical functions, avoiding changing state and mutable data.
- Emphasizes pure functions (same input always produces same output, no side effects).
- Key concepts: immutability, first-class functions, higher-order functions (`map`, `filter`, `reduce`), and function composition.
- Languages like Haskell, Lisp, and Erlang are functionally-first. Python supports functional features like lambda functions, comprehensions, and functional tools.
- Advantages include easier testing, debugging, and parallel execution due to stateless design.

Paradigm Blending in Modern Development

Few modern languages adhere strictly to a single paradigm. Languages like Python, JavaScript, and C# are **multi-paradigm**, incorporating features from object-oriented, functional, and imperative programming. This hybrid approach provides developers with flexibility, empowering them to select the most effective tools and patterns for each specific task, reflecting the practical demands of real-world software development.

Object Oriented Programming - Class

Contents

1. [What are Classes?](#)
2. [Class Anatomy](#)
3. [Creating and Using Objects](#)
4. [Test Harness](#)
5. [Special Methods \(Magic Methods\)](#)
6. [Constructor](#)
7. [Instance vs Class Attributes](#)
8. [Methods Types](#) (Instance, Class, Static)

Classes and Objects

1. A class is a software unit that describe the properties/states and behavior/actions of an entity
2. It is a template or blueprint for creating an object.
3. You may create multiple objects based on a single class definition
4. Each object will have its own distinct states

Class = Blueprint

Object = Instances of that blueprint

Class definition

- The following code defines a basic Account class
- It does not contain any methods or attributes
- It is used to demonstrate the basic format of a class in Python
- The class does not explicitly inherit from any other class

```
class Account:  
    pass
```

Creating and Using Object

To create an object (instance), you instantiate the class:

- The fundamentals test that a class must pass is
 - Instantiating the class i.e. to create an object based on the class
 - Invoking the methods and examining the object afterwards

The code to test the class is normally referred to as the **Test Harness**

```
#test harness to exercise the above code
```

Lecture 10

```
#the following code creates an instance of the Account class
#and assigns the reference to the variable obj
obj = Account()

print(obj)          # This will print the object representation of the
instance.           # At this point the output will not be very
                    # informative,
                    # later we will see how to make it more informative

print(type(obj))    # This will print <class '__main__.Account'>
indicating the type of the object
```

Adding Attribute Dynamically

Python is a dynamic language. You can attach attributes you an object at runtime

```
#the following code dynamically adds three attributes to the obj instance
#the attributes are balance, holder, and number and are assigned values of
100, 'narendra', and '1234'
obj.balance = 100
obj.holder = 'narendra'
obj.number = '1234'

#the following code prints the values of the attributes
print(f'balance: {obj.balance}')
print(f' holder: {obj.holder}')
print(f' number: {obj.number}')
```

Special Methods (Magic Methods)

1. `__init__(self, parameters)` -> Constructor (runs immediately after the an object is created).
2. `__str__(self)` -> String representation (controls what `print(obj)` shows)

The following illustrates a better class definition. Note the following:

1. A docstring: This is a string that briefly describes the class. It is used in python internal documentation system

```
# a basic Account class
class Account:
    '''A simple class to represent a bank account with a holder name, account
    number, and balance.
    This class allows you to create an account with a holder's name, account
    number, and an optional balance.

    Attributes:
    holder (str): The name of the account holder.
    number (str): The account number.
```

Lecture 10

`balance (float)`: The current balance of the account, defaulting to 0 if not specified.

Methods:

`__init__(name, number, balance=0)`: Initializes the account with a holder name, account number, and an optional balance.

```
'''  
  
def __init__(self, number, name, balance = 500) -> None:  
    '''This constructor initialize an Account object with a holder name,  
account number, and an optional balance.  
    If no balance is provided, it defaults to 0.'''  
    self.holder = name  
    self.number = number  
    self.balance = balance  
  
def __str__(self) -> str:  
    amt = f'${self.balance:,.2f}'  
    return f'{self.number} {self.holder:>10} {amt:>12}'  
  
# test harness  
first = Account('1234', 'Narendra')  
print(f'    first: {first}')  
  
second = Account('1235', 'Ilia', 5_000)  
print(f'    second: {second}')  
  
print(f'anonymous: {Account('1236', 'Mehrdad', 1_234)}')
```

More special Methods

```
class Professor:  
    def __init__(self, name, service):  
        self.name = name  
        self.service = service  
  
    def __str__(self):  
        return f'{self.name} ({self.service}yrs)'  
  
    def __eq__(self, value):  
        if isinstance( value, Professor) :  
            # return self.name == value.name and self.service ==  
value.service  
            return self.service == value.service  
        return False  
  
    def __lt__(self, value):  
        if isinstance(value, Professor):  
            return self.service < value.service  
        return False  
  
    def __gt__(self, value):  
        if isinstance( value, Professor):  
            return self.service > value.service
```

Lecture 10

```
        return False

obj1 = Professor('Narendra', 3)
obj2 = obj1

obj3 = Professor('Hao', 6)
obj4 = Professor('Hao', 6)

obj5 = Professor('Ilia', 3)

obj6 = Professor('Yin', 10)

print(f'{obj1} \\'is\\' {obj2} -> {obj1 is obj2}')
print(f'{obj1} \\'==\\' {obj2} -> {obj1 == obj2}')
print(f'{obj1} \\'==\\' {obj5} -> {obj1 == obj5}')

print(f'{obj3} \\'is\\' {obj4} -> {obj3 is obj4}')
print(f'{obj3} \\'==\\' {obj4} -> {obj3 == obj4}')

print(f'{obj3} \\'<\\' {obj5} -> {obj3 < obj5}')
print(f'{obj3} \\'>\\' {obj5} -> {obj3 > obj5}')
print(f'{obj6} \\'<\\' {obj3} -> {obj6 < obj3}')
print(f'{obj6} \\'>\\' {obj3} -> {obj6 > obj3}')
Narendra (3yrs) 'is' Narendra (3yrs) -> True
Narendra (3yrs) '==' Narendra (3yrs) -> True
Narendra (3yrs) '==' Ilia (3yrs) -> True
Hao (6yrs) 'is' Hao (6yrs) -> False
Hao (6yrs) '==' Hao (6yrs) -> True
Hao (6yrs) '<' Ilia (3yrs) -> False
Hao (6yrs) '>' Ilia (3yrs) -> True
Yin (10yrs) '<' Hao (6yrs) -> False
Yin (10yrs) '>' Hao (6yrs) -> True
```

Instance Methods

These are methods that are attached to objects of this class that modify or query their state. You use the dot operator on an object to access them.

```
# an even more useful class
class Account:
    '''A simple class to represent a bank account with a holder name, account
    number, and balance.
    This class allows you to create an account with a holder's name, account
    number, and an optional balance.

    Attributes:
        holder (str): The name of the account holder.
        number (str): The account number.
        balance (float): The current balance of the account, defaulting to 0 if
        not specified.

    Methods:
        __init__(name, number, balance=0): Initializes the account with a holder
        name, account number, and an optional balance.
```

Lecture 10

deposit(self, amount) -> None: Increases the balance by the amount specified by the argument

withdraw(self, amount) -> None: Decreases the balance by the amount specified by the argument

```
def __str__(self) -> str: Returns a string of this object

'''
def __init__(self, name: str, number: str, balance: float = 0) -> None:
    self.holder = name
    self.number = number
    self.balance = balance

def deposit(self, amount: float) -> None:
    '''To increase the balance'''
    self.balance += amount

def withdraw(self, amount: float) -> None:
    '''To decrease the balance'''
    self.balance -= amount

def __str__(self) -> str:
    return f'{self.number} {self.holder} ${self.balance:,.2f}'

# test harness
acct = Account('1234', 'narendra', 200.)
print(acct)

amt = 500.0
print(f'Deposit ${amt:,.2f}')
acct.deposit(amt)

amt = 150.0
print(f'Withdraw ${amt:,.2f}')
acct.withdraw(amt)
print(acct)
#problems with the above class

acct.balance = 1_000_000
print(acct)
# a better class
# an even more useful class
class Account:
    def __init__(self, name, number, balance = 0) -> None:
        self.__holder = name
        self.__number = number
        self.__balance = balance

    def deposit(self, amount) -> None:

        self.__balance += amount
```


Lecture 10

```
def withdraw(self, amount) -> None:

    self.__balance -= amount

def __str__(self) -> str:
    return f'{self.__number} {self.__holder} ${self.__balance:,.2f}'

@property
def balance(self):
    return self.__balance

# test harness
acct = Account('1234', 'narendra', 200.)
print(acct)

amt = 500.0
print(f'Deposit ${amt:,.2f}')
acct.deposit(amt)

amt = 150.0
print(f'Withdraw ${amt:,.2f}')
acct.withdraw(amt)
print(acct)

# acct.balance = 1_000_000
acct.__balance = 1_000_000
print(acct)
print(acct.__balance)
```

Class variables

```
#the following code defines a class named Instructor
#it has a class attribute 'name' set to 'Ilia'
class Instructor:
    name = 'Ilia'
print(Instructor.name) #the dot notation is used to access class attributes

mayy = Instructor() #creates a instance of Instructor
print(mayy.name) # object mayy does not have an attribute name
# so it looks for that attribute in the class
# it finds one and then print 'Ilia'

hao = Instructor() #creates a new instance of Instructor
print(hao.name) #again hao, does not have name, so it
# fetches the attribute in the class
# and print 'Ilia'

hao.name = 'Hao Lac' # sets the name attribute to 'Hao Lac'
print(hao.name) # prints 'Hao Lac'

print(mayy.name) #prints 'Ilia'
```

Lecture 10

```
Instructor.name = 'Arben Tapia' # Sets the class attribute to Arben Tapia
print(hao.name)                  #prints 'Hao Lac'

print(mayy.name)                 #prints 'Arben Tapia'
class Student:
    '''Represents a student in Centennial College.'''

    # Class attributes - shared by all students
    school_name = 'Centennial College'
    total_students = 0
    sin = 100_000

    def __init__(self, name, major):
        '''Initialize a new student.'''
        self.name = name                # Instance attribute
        self.student_id = f'STU{Student.sin}' # uses the class attribute
        Student.sin += 1                # update for the next student
        self.major = major              # Instance attribute
        self.grades = []                # Instance attribute
        self.enrolled_courses = []      # Instance attribute

        # Update class attribute
        Student.total_students += 1

    def add_grade(self, course, grade):
        '''Add a grade for a specific course.'''
        self.grades.append({'course': course, 'grade': grade})

    def calculate_gpa(self):
        '''Calculate the student's GPA.'''
        if not self.grades:
            return 0.0

        total_points = sum(grade['grade'] for grade in self.grades)
        return round(total_points / len(self.grades), 2)

    def enroll_course(self, course_name):
        '''Enroll student in a course.'''
        if course_name not in self.enrolled_courses:
            self.enrolled_courses.append(course_name)
            print(f'{self.name} enrolled in {course_name}')
        else:
            print(f'{self.name} is already enrolled in {course_name}')

    def get_info(self):
        '''Return formatted student information.'''
        return {
            'name': self.name,
            'id': self.student_id,
            'major': self.major,
            'grades': self.grades,
            'gpa': self.calculate_gpa(),
        }
```

Lecture 10

```
        'courses': self.enrolled_courses,
        'school': Student.school_name
    }
help(Student)
import pprint

# test harness
alice = Student('Alice Johnson', 'Software Engineering Technologist')
bob = Student('Bob Smith', 'Game - Programming')

alice.add_grade('Python Programming', 92)
alice.add_grade('Software Engineering Fundamentals', 88)
alice.enroll_course('Web Page Development')

print(f'Alice\'s GPA: {alice.calculate_gpa()}')
print(f'Total students: {Student.total_students}')

pprint.pp(alice.get_info(), indent=2)

pprint.pp(bob.get_info(), indent=2)
```

Static Methods

These methods do not rely on class or instance data to work.

They are decorated with `@staticmethod`.

The class below, is just an assembly of string methods contained in a class.

```
class StringUtils:
    '''Utility class for string operations.'''

    @staticmethod
    def reverse_words(text):
        '''Reverse the order of words in a string.'''
        return ' '.join(text.split()[::-1])

    @staticmethod
    def is_palindrome(text):
        '''Check if text is a palindrome (ignoring spaces and case).'''
        cleaned = ''.join(text.lower().split())
        return cleaned == cleaned[::-1]

    @staticmethod
    def count_words(text):
        '''Count words in a text.'''
        return len(text.split())

    @staticmethod
    def capitalize_words(text):
        '''Capitalize first letter of each word.'''
        return ' '.join(word.capitalize() for word in text.split())

# Test Harness
# To access a static method, use the dot operator on the class name
to_test = 'Hello World Python'
```

Lecture 10

```
print(f'        Original: {to_test}')
```

```
print(f'        Reversed: {StringUtils.reverse_words(to_test)}')
```

```
to_test = 'A man a plan a canal Panama'
```

```
print(f'        Original: {to_test}')
```

```
print(f' Is palindrome: {StringUtils.is_palindrome(to_test)}')
```

```
to_test = 'This is a test sentence'
```

```
print(f'        Original: {to_test}')
```

```
print(f'    Word count: {StringUtils.count_words(to_test)}')
```

```
to_test = 'mark joseph carney'
```

```
print(f'        Original: {to_test}')
```

```
print(f'    Capitalise: {StringUtils.capitalize_words(to_test)}')
```

```
class Professor:
```

```
    def __init__(self, name):
```

```
        self.name = name
```



```
arben = Professor('Arben')
```

```
print(arben.name)
```



```
faculty1 = Professor('Jake')
```

```
print(faculty1.name)
```